

This document presents a **simplified, buildable version** of the MYLIFE architecture — trimmed to 5 core microservices, removing complexity like Redis caching, RabbitMQ messaging, and extra features. Everything here can be built and deployed by a small team.

What We Simplified

8 services → 5	Merged Women's Health + Family into one service; merged Emergency into Medical Records; dropped standalone Verification service (handled inside Auth).
RabbitMQ removed	Direct REST calls between services instead of async messaging. Simpler to build, debug, and deploy.
Redis removed	PostgreSQL handles sessions via Supabase Auth. No separate cache layer needed for MVP.
AI/OCR simplified	One optional AI endpoint using Claude API only. No Tesseract, PaddleOCR, or GPT/Gemini stack.
Deployment simplified	Docker Compose on a single VPS (Railway/Render). No Kubernetes, no CDN setup for MVP.

Technology Stack

Frontend	Backend
• React.js + TypeScript • Tailwind CSS • Axios + React Query • Supabase Auth SDK • Deployed on Vercel	• FastAPI (Python) • Supabase PostgreSQL • Supabase Storage (files) • JWT authentication • Deployed on Railway/Render

No Redis. No RabbitMQ. No separate message broker. Supabase handles auth, database, file storage, and real-time. FastAPI handles all business logic. This is enough for MVP.

System Architecture Overview

[illegible]

The 5 Microservices

Service 01	Authentication Service — port 8001
Purpose	Handles all user identity: register, login, JWT tokens, roles, and permissions.
Responsibilities	• User registration & login • JWT token issue & refresh • Role management (Patient, Doctor, Admin) • Password reset via email
DB Tables	users, roles, sessions
API Endpoints	POST /auth/register POST /auth/login POST /auth/refresh POST /auth/logout GET /auth/me

Service 02		Medical Records Service — port 8002	
Purpose	Core of the platform. Stores and manages all health records, QR sharing, and emergency profiles.		
Responsibilities	• Store medical records & diagnoses • Medication & allergy history • QR code sharing • Emergency profile (offline-safe) • SOS alert trigger		
DB Tables	medical_records, diagnoses, medications, shared_records, emergency_profiles		
API Endpoints	POST /records GET /records/{id} PUT /records/{id} DELETE /records/{id} POST /records/share-qr GET /emergency/profile/{userId}		

Service 03	Family & Profile Service — port 8003
Purpose	Manages linked family accounts, caregivers, children, and women's health tracking.
Responsibilities	• Link/unlink family accounts • Caregiver & child profiles • Menstrual & pregnancy tracking • Permission management • Elderly parent management
DB Tables	linked_accounts, family_profiles, menstrual_cycles, pregnancy_records, caregiver_permissions
API Endpoints	POST /family/link GET /family/members DELETE /family/unlink POST /health/cycle GET /health/cycle POST /health/pregnancy

Service 04	AI Processing Service — port 8004
Purpose	Optional but powerful. Uses Claude API to extract structured data from uploaded medical documents.
Responsibilities	• Accept uploaded PDF/image • Send to Claude API for extraction • Return structured medical data • Store extraction results • Confidence scoring
DB Tables	uploaded_documents, extracted_reports
API Endpoints	POST /ai/process (upload file) GET /ai/results/{id} POST /ai/summary

Service 05	Notification Service — port 8005
Purpose	Sends emails and push notifications for key events. Called directly by other services.
Responsibilities	• Email notifications (SendGrid/Resend) • Push notifications (Firebase FCM) • Appointment & pregnancy reminders • Emergency alerts • Verification complete alerts
DB Tables	notifications, notification_logs

API Endpoints

POST /notify/email

POST /notify/push

POST /notify/reminder

Database Architecture

All services use **Supabase PostgreSQL** with separate schemas. Each service reads and writes only its own schema — no cross-service DB calls.

Schema	Tables
auth_schema	users, roles, sessions, permissions
medical_schema	medical_records, diagnoses, medications, allergies, shared_records, emergency_profiles
family_schema	linked_accounts, family_profiles, menstrual_cycles, pregnancy_records, caregiver_permissions
ai_schema	uploaded_documents, extracted_reports, extraction_logs
notification_schema	notifications, notification_logs

File Storage: Supabase Storage Buckets — medical-docs, profile-pictures. No separate file server needed.

Inter-Service Communication

All communication is **synchronous REST (HTTP)**. Simple, debuggable, no message broker needed.

Who Calls Who	Why
Medical → Auth	Validate JWT token for protected routes
Medical → Notification	POST /notify/push after record upload
Family → Auth	Check user roles for family permission
AI → Medical	Return extracted data, stored in medical_schema
AI → Notification	POST /notify/email when extraction complete

Security (Simplified)

Authentication & Transport	Authorization & Privacy
----------------------------	-------------------------

- JWT tokens (access + refresh)
 - Supabase Row Level Security
 - HTTPS on all endpoints
 - CORS enforced at gateway
- Role-based access control
 - Input validation via Pydantic
 - Audit log table per service
 - Consent before record sharing

Project Folder Structure

```
mylife/ ├── frontend/ │   ├── src/ │   │   ├── components/ │   │   ├── pages/ │   │   └── ... │   └── ... │   └── services/ ← API call functions │   │   ├── hooks/ │   │   ├── package.json │   │   └── ... │   └── services/ │   │   ├── auth-service/ │   │   │   ├── main.py │   │   │   ├── models.py │   │   │   └── ... │   │   ├── routes.py │   │   ├── schemas.py │   │   ├── requirements.txt │   │   ├── medical-service/ │   │   ├── family-service/ │   │   ├── ai-service/ │   │   ├── notification-service/ │   │   └── ... │   └── gateway/ │   │   ├── nginx.conf ← Route /auth → 8001, /records → 8002 etc. │   │   └── ... │   └── docker-compose.yml │   └── .env
```

Docker Compose Setup

```
services: auth-service: build: ./services/auth-service ports: ["8001:8001"] env_file: .env medical-service: build: ./services/medical-service ports: ["8002:8002"] env_file: .env family-service: build: ./services/family-service ports: ["8003:8003"] env_file: .env ai-service: build: ./services/ai-service ports: ["8004:8004"] env_file: .env notification-service: build: ./services/notification-service ports: ["8005:8005"] env_file: .env gateway: image: nginx:alpine ports: ["80:80"] volumes: ["/gateway/nginx.conf:/etc/nginx/nginx.conf"] depends_on: [auth-service, medical-service, family-service, ai-service, notification-service]
```

Key Flow: Upload & Process Medical Record

Step	Action
Step 1	Patient uploads PDF via React frontend
Step 2	Request hits NGINX gateway → routed to Medical Records Service (8002)
Step 3	Medical Service validates JWT with Auth Service (8001)
Step 4	File saved to Supabase Storage; metadata saved to medical_schema
Step 5	Medical Service calls AI Service (8004) with file URL

Step 6	AI Service sends to Claude API → gets structured data back
Step 7	Extracted data saved to ai_schema.extracted_reports
Step 8	AI Service calls Notification Service (8005) → email sent to patient
Step 9	Patient refreshes app → sees verified, extracted record

MVP Build Scope

Build for MVP	Skip for V1
• Auth Service (full) • Medical Records CRUD • QR code sharing • Family account linking • Basic women's health tracking • AI document extraction (Claude) • Email notifications	• Redis / RabbitMQ • Pharmacy integration • Insurance integration • Full appointment booking • Government API integration • Multi-language OCR (Tesseract) • Advanced AI diagnosis

Recommended Deployment

Layer	Tool	Why
Frontend	Vercel	Free tier, auto-deploy from GitHub
Backend Services	Railway or Render	Docker container hosting, easy env vars
Database	Supabase	Managed PostgreSQL + Auth + Storage
File Storage	Supabase Storage	Built-in, no S3 setup needed
Domain / SSL	Cloudflare	Free SSL, DNS management
CI/CD	GitHub Actions	Push → test → build Docker → deploy